

五子棋对弈机械臂运动控制技术报告

基于 gobang_ws 工程实现

郭政和 3230100923 生工食品学院

2026 年 7 月

目录

1	系统概述	3
2	坐标系定义	3
2.1	机械臂基座系 <code>arm_base_link</code>	3
2.2	末端执行器系 <code>arm_tool0</code>	3
2.3	棋盘坐标系（基座系下的平面映射）	4
2.4	世界系与机械臂系对齐问题	4
3	DH 参数与连杆几何	4
3.1	六轴 DH 表	5
4	正运动学	5
5	逆运动学	6
5.1	解析逆解（8 组解）	6
5.2	解的选择策略	6
5.3	边缘格点与高度补偿	6
6	关节限位与运动规划	7
6.1	软件关节限位（rad）	7
6.2	等价角与最短路径	7
6.3	笛卡尔运动实现	7
7	零位标定	7
7.1	绝对零位（电机闪存）	7
7.2	相对零位（软件）	8

目录	2
7.3 回零验证	8
8 棋盘落点生成与执行	8
8.1 离线生成	8
8.2 在线执行 API	8
9 末端姿态与高度控制	9
9.1 保持 Z 轴竖直	9
9.2 末端距桌面高度	9
10 工程问题分析与思考	9
10.1 电机角度与零位“莫名其妙变了”	9
10.2 无限位导致 2π 缠绕与卡死	9
10.3 关节松动与落点精度	10
10.4 机械臂系与世界系/棋盘对齐（最头痛）	10
10.5 无限位时的碰撞路径（桌下/软管）	10
10.6 棋盘与臂工作空间—不可达格点	11
10.7 末端竖直与高度小结	11
11 ROS 接口汇总	11
12 标定与调试命令速查	12
13 结论	12

1 系统概述

本项目为人机对弈五子棋系统：人类在 10×10 棋盘上落子，机械臂负责从零点取子并放置到 AI 指定格心。机械臂相关软件位于 `gobang_ws` 工作空间，核心包如下：

表 1: 机械臂相关 ROS 包

包名	路径	职责
<code>six_dof_arm_driver</code>	<code>src/six_dof_arm_driver/</code>	CAN 总线驱动、关节/笛卡尔 A TF 发布
<code>robot_gobang_motion</code>	<code>src/robot_gobang_motion/</code>	棋盘标定、逆解表生成、落子运动
<code>vacuum_gripper_driver</code>	<code>src/vacuum_gripper_driver/</code>	气动真空吸盘 (CAN 0x21)
<code>robot_gobang_calibration</code>	<code>src/robot_gobang_calibration/</code>	手眼静态 TF

硬件配置： Robodyno 六自由度协作机械臂，CAN ID 为 `0x10--0x15`（十进制 16–21）；末端为气动真空吸盘，连杆 156 由原厂 45 mm 加长至 100 mm（含吸盘 55 mm）。

运动流程：

1. 使能并关节空间回零 $\mathbf{q}_{\text{HOME}} = [0, 0, 0, 0, 0, 0]^T$ （竖直姿态）；
2. 真空吸盘吸合，人工喂子；
3. 笛卡尔空间运动至目标格心 (x, y, z) ，姿态固定为吸盘竖直向下；
4. 真空释放，棋子落盘；
5. 可选回零。

2 坐标系定义

2.1 机械臂基座系 `arm_base_link`

以机械臂底座安装面为原点， Z 轴竖直向上， X 轴指底座前方， Y 轴按右手定则确定。竖直零位（ $\mathbf{q} = \mathbf{0}$ ）时，末端 TCP 名义位置为：

$$\mathbf{p}_{\text{HOME}} = (0, -0.175, 0.62)^T \quad (\text{m}) \quad (1)$$

该值由 DH 正运动学计算（`arm_fk.py`），容差 $\pm(0.03, 0.03, 0.04)$ m。

2.2 末端执行器系 `arm_tool0`

TCP 位于第 6 关节法兰延长 156 处。落子时要求吸盘始终垂直向下，即工具 Z 轴与基座 $-Z$ 对齐。工程采用固定 RPY 姿态：

$$\Phi_{\text{tool}} = (\text{roll}, \text{pitch}, \text{yaw}) = (-\pi, 0, -\pi/2) \quad (2)$$

注意： 不可使用 $\text{yaw} = 0$ ，否则手腕会额外扭转 90° ，导致吸盘不竖直。

2.3 棋盘坐标系（基座系下的平面映射）

棋盘为 10×10 格，格心间距 $\Delta = 0.025 \text{ m}$ (25 mm)。左下角（1 行 1 列）在基座系中的坐标由 `grid_coords.py` 定义：

$$\mathbf{p}_{\text{corner}} = \text{CORNER_XY} = (0.200, -0.150)^T \quad (\text{m}) \quad (3)$$

格心 (lr, fb) (0-based 列、行索引) 映射为：

$$x = x_0 + \text{fb} \cdot \Delta \quad (4)$$

$$y = y_0 + \text{lr} \cdot \Delta \quad (5)$$

其中 x_0, y_0 为 `CORNER_XY`。行列约定：行 fb 沿 $+X$ ，列 lr 沿 $+Y$ ；对外接口 1-based，左下角为 (1,1)。

落子高度：

$$z_{\text{place}} = \text{PLANE_Z} = 0.003 \text{ m} \quad (6)$$

（吸盘底面距基座水平面约 3 mm；实际需根据棋子厚度微调。）

2.4 世界系与机械臂系对齐问题

相机、棋盘、底座三者的空间对齐是本系统最大难点。机械臂只认 `arm_base_link`；棋盘角点 `CORNER_XY` 必须在基座系中实测标定。若底座相对棋盘发生旋转（例如每次搬运后“左边”方向变化），须：

1. 将臂摆至竖直零位，确认 $-Y$ 为臂伸展方向；
2. 观察棋盘相对基座的方位（前/左/右）；
3. 修改 `grid_coords.py` 中 `CORNER_XY`，重新运行 `gen_board_positions.py`。

手眼标定(可选)通过 `robot_gobang_calibration` 发布静态 TF: `animation_base_link` $\rightarrow$ `camera_color_optical_frame`，用于视觉模块，不直接参与落子笛卡尔坐标（落子坐标硬编码在基座系）。

3 DH 参数与连杆几何

采用标准 DH (Modified/Classic 与 Robodeno 库一致)，单关节变换：

$$\mathbf{T}_i^{i-1} = \text{Trans}_z(\theta_i) \text{Trans}_z(d_i) \text{Trans}_x(a_i) \text{Rot}_x(\alpha_i) \quad (7)$$

Robodeno 封装为 `dh_matrix( $\alpha, a, d, \theta$ )`。

3.1 六轴 DH 表

表 2: Robodyno 六轴协作臂 DH 参数 (six_dof_arm_param.yaml)

连杆	α_{i-1}	a_{i-1} (m)	d_i (m)	θ_i	说明
1	$\pi/2$	0	$l_{01} = 0.175$	q_1	基座 $\rightarrow$ 肩
2	0	$-l_{12} = -0.185$	0	$q_2 - \pi/2$	肩 $\rightarrow$ 肘
3	0	$-l_{23} = -0.185$	0	$-q_3$	肘 $\rightarrow$ 腕
4	$\pi/2$	0	$l_{34} = 0.075$	$q_4 - \pi/2$	腕 1
5	$-\pi/2$	0	$l_{45} = 0.075$	q_5	腕 2
6	0	0	$l_{56} = 0.100$	q_6	法兰 $\rightarrow$ TCP

连杆总长: $l_{01} + l_{12} + l_{23} + l_{34} + l_{45} + l_{56} \approx 0.795 \text{ m}$ 。

4 正运动学

末端齐次变换:

$$\mathbf{T}_0^6 = \mathbf{T}_0^1 \mathbf{T}_1^2 \mathbf{T}_2^3 \mathbf{T}_3^4 \mathbf{T}_4^5 \mathbf{T}_5^6 \quad (8)$$

展开 (与 arm_fk.py、gen_board_positions.py 一致):

$$\mathbf{T}_0^1 = \text{DH}(\pi/2, 0, l_{01}, q_1) \quad (9)$$

$$\mathbf{T}_1^2 = \text{DH}(0, -l_{12}, 0, q_2 - \pi/2) \quad (10)$$

$$\mathbf{T}_2^3 = \text{DH}(0, -l_{23}, 0, -q_3) \quad (11)$$

$$\mathbf{T}_3^4 = \text{DH}(\pi/2, 0, l_{34}, q_4 - \pi/2) \quad (12)$$

$$\mathbf{T}_4^5 = \text{DH}(-\pi/2, 0, l_{45}, q_5) \quad (13)$$

$$\mathbf{T}_5^6 = \text{DH}(0, 0, l_{56}, q_6) \quad (14)$$

TCP 位置 $\mathbf{p} = [x, y, z]^\top$ 与姿态 (roll, pitch, yaw) 由 $\mathbf{T}_0^6$ 分解得到。

验证: gen_board_positions.py 在生成 100 格关节表后, 对每组关节角做 FK 回代, 要求位置误差 $< 10 \text{ mm}$ 。

5 逆运动学

5.1 解析逆解（8 组解）

目标位姿 $T_0^6 = T(x, y, z) \text{Rot}(\text{roll}, \text{pitch}, \text{yaw})$ 。先求腕心 $\mathbf{p}_{05}$ （第 5 关节原点）：

$$\mathbf{p}_{05} = T_0^6 \begin{bmatrix} 0 \\ 0 \\ -l_{56} \\ 1 \end{bmatrix} \quad (15)$$

关节 1：令 $\phi_1 = \text{atan2}(p_y, p_x)$ ， $\phi_2 = (l_{34}/\|\mathbf{p}_{05,xy}\|)$ （需 $\|\mathbf{p}_{05,xy}\| \geq l_{34}$ ），得两组：

$$q_1 = \pi/2 + \phi_1 \mp \phi_2 \quad (16)$$

关节 5、6：由 T_6^1 几何关系求 $q_5 = (\cdot)$ ， $q_6 = \text{atan2}(\cdot)$ 。

关节 2、3：在肩-肘平面用余弦定理：

$$c_3 = \frac{\|\mathbf{p}_{13}\|^2 - l_{12}^2 - l_{23}^2}{2l_{12}l_{23}} \quad (17)$$

$|c_3| \leq 1$ 方有解；进而 q_2, q_3 ； q_4 由 T_3^4 反解。

实现见 `gen_board_positions.py` 中 `ik_all()`，最多 8 组解析解。

5.2 解的选择策略

对每格落点，在可行解中选与参考关节角 $\mathbf{q}_{\text{ref}}$ 欧氏距离最小者：

$$J = \sum_{i=1}^6 (q_i - q_{\text{ref},i})^2 + \lambda \cdot \mathbb{I}(|q_1| > \pi/2) \quad (18)$$

$\lambda = 8$ 惩罚基座翻转（ $q_1 \approx \pm\pi$ ），避免臂绕到棋盘对面。

运行时 `six_dof_arm_node.py` 的 `_pick_best_ik()` 在 8 个 `sol_id` 间选代价最小且在软件限位内的解。

5.3 边缘格点与高度补偿

部分格点（尤其远角）可能无解析解。生成脚本策略：

- x 方向内缩 0–30 mm；
- z 从 `PLANE_Z` 向下搜索最多 150 mm；
- 悬停高度 $z_{\text{hover}} = z + \text{APPROACH_DZ}$ （默认 60 mm）。

6 关节限位与运动规划

6.1 软件关节限位 (rad)

关节	下限	上限	约等于
q_1	$-\pi$	π	$\pm 180^\circ$
q_2	$-2\pi/3$	$2\pi/3$	$\pm 120^\circ$
q_3	$-7\pi/8$	$7\pi/8$	$\pm 157.5^\circ$
q_4	$-9\pi/10$	$9\pi/10$	$\pm 162^\circ$
q_5	$-\pi$	π	$\pm 180^\circ$
q_6	$-\pi$	π	$\pm 180^\circ$

标定脚本 `calibrate_arm_full.py` 将上述限位写入电机；超出限位的 IK 解被丢弃。

6.2 等价角与最短路径

旋转关节存在 $\theta \equiv \theta + 2k\pi$ 。回零时必须到达字面 0，故关节 Action 使用 `literal=True`，禁止 $\pm 2\pi$ 改写。其他运动由 `_plan_joint_target()` 在 $[\theta - 4\pi, \theta + 4\pi]$ 中选距当前角最近且在限位内的等价角。

6.3 笛卡尔运动实现

`cartesian_via_joint_ik: true`: 笛卡尔 Action 先 IK 得关节角，再关节插值执行（非真笛卡尔直线）。插值周期 `joint_motion_period_s=0.12s`，速度上限 `0.8rad/s`。

7 零位标定

7.1 绝对零位（电机闪存）

流程（`calibrate_arm_full.py`）：

1. 手动将臂摆至完全竖直，停止 ROS 驱动；
2. 读取各轴 `get_abs_pos()`，若 $|\theta| > 2\pi$ 提示异常；
3. `init_abs_pos(0) + save()` 写入闪存；
4. `init_pos(0)` 设相对零位；

5. 设置速度限制与软件限位;

6. 重算 board_positions_data.py (100 格)。

标定记录示例 (arm_calibration.yaml) 显示标定前 J5、J6 绝对角约 6.2rad、-5.86rad (累计多圈), 标定后均为 0。

7.2 相对零位 (软件)

calibrate_vertical_home.py: 竖直姿态下 enable 且 recalibrate_relative_zero_on_enable= 调用 init([0]*6), 不移动, 仅将当前姿态定义为软件零。

关键参数:

- recalibrate_relative_zero_on_enable: false (正常运行) —— enable 时不覆盖闪存零点;
- 若为 true, 每次 enable 把当前任意姿态当零点, 导致 “零点莫名其妙变了”。

7.3 回零验证

verify_return_home.py: 从任意姿态发 HOME 指令, 检查 $|q_i| < 0.08\text{rad}$ 。

diagnose_arm_zero.py: 联合检查关节读数、FK 与 HOME_TCP_XYZ 一致性。

8 棋盘落点生成与执行

8.1 离线生成

python3 gen_board_positions.py 输出 board_positions_data.py, 每格含:

- xyz、hover_xyz: 落点与悬停笛卡尔坐标;
- joints、hover_joints: 对应关节角;
- ik_sol_id: 选用的解析解编号 (-1 表示无解)。

8.2 在线执行 API

ArmMotionAPI (arm_motion_api.py):

- move_home(): enable $\rightarrow$ 关节回 $[0]^6 \rightarrow$ 多次验证;
- feed_at_home(): 回零后真空吸合;
- place_at_grid(lr,fb): 笛卡尔至格心, 释放真空。

落子高度由 PLANE_Z 与可选 place_z_offset_m 控制; 到位后 FK 校验, 误差 $> 20\text{mm}$ 报警。

9 末端姿态与高度控制

9.1 保持 Z 轴竖直

六轴臂一般不能在全域保持同一工具姿态的同时覆盖所有位置（6-DOF 无冗余）。本方案通过固定目标 RPY $(-\pi, 0, -\pi/2)$ 在 IK 阶段约束腕部，使吸盘法向近似 $-\mathbf{e}_z$ 。

若某格 IK 无解，可微调 (x, y, z) 或略改 roll/yaw（工程上优先调 z 与 x 内缩）。

9.2 末端距桌面高度

设基座安装面为 $z = 0$ 参考，棋盘平面高度 z_b ，棋子高度 h_{piece} ，吸盘压缩量 δ ：

$$z_{\text{place}} = z_b + h_{\text{piece}} + \delta_{\text{gripper}} \quad (19)$$

当前 PLANE_Z=0.003 为经验值；更换棋子或棋盘厚度后须重测并更新 grid_coords.py，再 sync_board_geometry.py。

悬停高度： $z_{\text{hover}} = z_{\text{place}} + 0.06 \text{ m}$ ，避免碰撞棋子。

10 工程问题分析与思考

本节结合现场调试经验，对主要故障模式给出机理分析与对策。

10.1 电机角度与零位“莫名其妙变了”

现象：断电重启或再次 enable 后，软件读数为 0 但 TCP 不在竖直，或落点系统性偏移。

机理：

1. **相对零与绝对零混淆：** recalibrate_relative_zero_on_enable=true 时 enable 调用 `init([0]*6)`，把当前姿态写入软件零点，而非闪存标定值。
2. **手动掰臂后未重标定：**失使能状态下转动关节，电机多圈计数与软件模型脱节。
3. **闪存未 save：**仅 `init_abs_pos` 未 `save()`，断电丢失。

对策：竖直姿态下运行 `calibrate_arm_full.py` 写闪存；正常运行保持 `recalibrate_relative_zero` enable 前确认臂已在竖直附近；用 `diagnose_arm_zero.py` 巡检。

10.2 无限位导致 2π 缠绕与卡死

现象：手动回零时某关节已转满 360° ，程序仍认为需再转一圈才能到目标，臂顶到机械限位动不了。

机理：旋转关节角 θ 与 $\theta + 2\pi$ 几何同姿但电机行程不同；无软件限位时 IK 可能选出“远路”解；`_plan_joint_target` 虽做最短路径，但目标本身在限位外则失败。

标定数据佐证：标定前 $J5 \approx 6.21 \text{ rad}$ ($\approx 356^\circ$)， $J6 \approx -5.86 \text{ rad}$ ($\approx -336^\circ$)，即已偏离 $[-\pi, \pi]$ 主值。

对策：

- 标定脚本写入官方六轴限位；
- 关节 Action 回零用 `literal=True`；
- 禁止在使能外大幅掰臂；若已缠绕，重新 `calibrate_arm_full.py`。

10.3 关节松动与落点精度

现象：使用一段时间后，同样程序落点偏几毫米至厘米，Repeatability 变差。

机理：关节间隙 $\Delta\theta$ 在连杆末端放大为 $\Delta p \approx L_{\text{eff}} \Delta\theta$ 。对 $L \sim 0.5 \text{ m}$ 、 $\Delta\theta \sim 0.01 \text{ rad}$ ， $\Delta p \sim 5 \text{ mm}$ 。

对策：紧固螺钉；定期重标 `CORNER_XY`；关键格点实测修正；考虑在 HOME 附近喂子减小行程。

10.4 机械臂系与世界系/棋盘对齐（最头痛）

现象：每次搬运底座或棋盘后，须“拎着臂竖直转 90° 看左边在哪”再重标。

机理：落点坐标硬编码在 `arm_base_link`；棋盘相对基座的平移 (x_0, y_0) 与旋转 ψ 任一变化即整体偏移。底座 $350 \times 350 \text{ mm}$ ，10 格棋盘 225 mm ，工作空间余量有限， 5° 旋转在远端可造成 $> 10 \text{ mm}$ 误差。

对策：

- 固定底座与棋盘相对位置，贴齐底座前边与左边（见 `grid_coords.py` 注释）；
- 改 `CORNER_XY` 而非改 IK；
- 可选：ArUco 测棋盘角点，在线修正（当前版本未启用闭环）。

10.5 无限位时的碰撞路径（桌下/软管）

现象：臂从棋盘上方走到 HOME 时，先钻到桌面以下，砸断前端气动软管或刮擦桌沿。

机理：关节空间插值不保证笛卡尔路径；IK 选解仅优化关节距离，不避障；无 $z_{\min}$ 约束。

对策：

- 软件限位缩小 q_2, q_3 范围，禁止大俯仰；
- 运动前必经 HOME 竖直姿态 (`from_home=true`)；
- 配置 `safe_transit_z`（可选中间抬升点 0.28 m ）；
- 物理上固定软管走线并加保护簧。

10.6 棋盘与臂工作空间—不可达格点

现象：四角/base 角格 `ik_sol_id=-1` 或运行时 IK 失败。

机理：六轴臂可达空间为环状壳体；棋盘近端/远端/侧边超出 $z_{\min}$ 或 $|q_1|$ 过大区域不可达； $\|p_{05,xy}\| < l_{34}$ 时解析 IK 无解。

对策：`gen_board_positions.py` 自动 x 内缩与 z 下调；缩小棋盘或前移 `CORNER_XY`；换 `default_ik_sol_id` 或手动指定 `sol_id`。

10.7 末端竖直与高度小结

固定 `TOOL_RPY = (- $\pi$ , 0, - $\pi/2$ )`； z 由 `PLANE_Z` 控制；换吸盘须同步改 156 并重算全表。

11 ROS 接口汇总

接口	类型	说明
<code>/six_dof_arm/enable</code>	<code>std_srvs/Trigger</code>	使能电机
<code>/six_dof_arm/joint_space_motion</code>	Action	6 关节角 (rad) + duration
<code>/six_dof_arm/cartesian_space_motion</code>	Action	目标 Pose + duration
<code>/six_dof_arm/joint_states</code>	<code>JointState</code>	当前关节角
<code>/vacuum_gripper/open</code>	Service	释放棋子
<code>/vacuum_gripper/close</code>	Service	吸附棋子

12 标定与调试命令速查

命令	用途
<code>python3</code> <code>calibrate_arm_full.py</code>	竖直姿态写电机闪存绝对零位，重算 100 格
<code>roslaunch robot_gobang_motion</code> <code>calibrate_vertical_home.py</code>	仅设软件零（不闪存）
<code>python3</code> <code>gen_board_positions.py</code>	由 DH+IK 生成 <code>board_positions_data.py</code>
<code>roslaunch robot_gobang_motion</code> <code>verify_return_home.py</code>	验证回零
<code>roslaunch robot_gobang_motion</code> <code>diagnose_arm_zero.py</code>	零点与 FK 诊断
<code>roslaunch robot_gobang_motion</code> <code>go_to_cell.py _row:=5</code> <code>_col:=5</code>	单格落子测试

13 结论

本系统采用 Robodino 标准 DH 模型与解析逆运动学，将 10×10 棋盘格心预计算为关节角表，运行时通过 ROS Action 执行“回零 → 笛卡尔落子”序列。精度的核心在于：（1）竖直绝对零位的闪存标定与禁止 enable 时覆盖零点；（2）关节软件限位防止 2π 缠绕；（3）CORNER_XY 与 PLANE_Z 与实物棋盘对齐；（4）156、TOOL_RPY 与真机末端一致。现场出现的零位漂移、卡死、落点偏移、碰撞软管等问题，均可从上述四类标定与约束机制溯源并系统性解决。